BÖLÜM 7 / MODULE 7

Red Hat OpenShift (OCP) – Kurumsal Kubernetes

Red Hat OpenShift (OCP) – Enterprise Kubernetes

Kurumsal bankaların ve dev telekomların tercihi: Standart K8s vs OpenShift, CoreOS, `oc` komutları, Routes, SCC güvenlik kısıtları ve S2I ile otomatik derleme.

The enterprise standard for banks and telcos: Vanilla K8s vs OpenShift, CoreOS, `oc` CLI, Routes, strict SCC security policies, and Source-to-Image (S2I) builds.

37

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Nedir? Standart K8s vs OpenShift

What is OpenShift? Vanilla Kubernetes vs Red Hat OpenShift

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Düz Kubernetes (Vanilla K8s), parçaları kutuda gelen bir yarış arabası şasisidir; motoru vardır ama koltukları, emniyet kemerini, klimayı, hatta kapı kollarını (Monitoring, Ingress, Registry, CI/CD) sanayiye gidip kendiniz tek tek taktırmak zorundasınız (Vahşi Batı!). Red Hat OpenShift ise fabrikadan zırhlı camları, deri koltukları, 24 saat yol yardımı ve tam güvenlik donanımıyla anahtar teslim çıkmış kurumsal bir Mercedes'tir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Vanilla Kubernetes is a DIY kit car chassis: you get an engine, but you must fabricate your own seats, seatbelts, AC, and mirrors (monitoring, ingress, registries, CI pipelines). Red Hat OpenShift is an armor-plated, fully assembled, factory-warrantied enterprise Mercedes delivered turnkey with 24/7 Red Hat enterprise support!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Red Hat OpenShift Container Platform (OCP), CNCF sertifikalı standart Kubernetes çekirdeği üzerine inşa edilmiş, tam teşekküllü kurumsal bir uygulama platformudur. Düz K8s'e ek olarak şunları yerleşik (out-of-the-box) getirir: 1) Yerleşik kurumsal Container Registry, 2) Prometheus/Grafana izleme yığını, 3) OpenShift Router (HAProxy tabanlı Route mekanizması), 4) Güçlü Web Konsolu (Developer ve Admin modları), 5) Otomasyonlu işletim sistemi (Red Hat Enterprise Linux CoreOS).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift is a CNCF-certified Kubernetes distribution wrapped with integrated enterprise services. Beyond the core K8s API, OCP pre-packages: an integrated OCI image registry, full-stack Prometheus/Alertmanager monitoring, automated OS provisioning via RHEL CoreOS, HAProxy routing, Git-to-deployment pipelines (BuildConfigs), and a unified developer-centric Web UI.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Standart Kubernetes CLI:
kubectl get pods

# OpenShift CLI (Tam geriye dönük uyumlu + ek süper güçler!):
oc get pods
oc status
oc whoami
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Düz Kubernetes için yazılmış standart YAML dosyalarının %99'u OpenShift üzerinde doğrudan çalışır. Ancak OpenShift'in varsayılan güvenlik kısıtlamaları (SCC) nedeniyle `root` olarak çalışan konteynerler anında engellenir!

🇬🇧 WATCH OUT

While 99% of standard K8s manifests run on OpenShift, containers requiring root privileges will be immediately rejected by OpenShift's strict security policies.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- OpenShift, Kubernetes motoru üzerine inşa edilmiş anahtar teslim kurumsal platformdur.
- Geliştirici odaklı araçlar, izleme ve imaj depoları yerleşik olarak gelir.
- Dünyadaki büyük bankaların ve telekomünikasyon şirketlerinin 1 numaralı tercihidir.

- OpenShift is a fully supported, production-ready enterprise distribution of Kubernetes.
- Built-in integrated registry, monitoring, build pipelines, and routing out of the box.
- The enterprise standard across Fortune 500 banks, airlines, and telecom leaders.

38

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Mimarisi: CoreOS, OKD ve Platform Operatörleri

OpenShift Architecture: CoreOS, OKD, and Platform Operators

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Geleneksel sunucularda işletim sistemini güncellemek mayın tarlasında yürümek gibidir; bir paket uyuşmaz, tüm sunucu çöker. OpenShift ise Red Hat CoreOS adı verilen 'değiştirilemez' (immutable) bir işletim sistemi kullanır. Düğümlerde SSH ile dosya değiştiremezsiniz; tüm işletim sistemi bir konteyner imajı gibi tek parça halinde atomik olarak güncellenir! Kümenin her organını da özel bir robot (Cluster Operator) yönetir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Upgrading standard Linux servers is like tiptoeing through a minefield; one conflicting library breaks everything. OpenShift runs on Red Hat Enterprise Linux CoreOS (RHCOS)—an immutable, container-optimized OS. You don't SSH into nodes to patch them; the entire OS updates atomically like an iPhone firmware update! Cluster Operators supervise every vital subsystem 24/7.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OpenShift 4 mimarisi iki temel kavrama dayanır: 1) RHCOS (Red Hat Enterprise Linux CoreOS): Salt-okunur kök dosya sistemi ve `ignition` ile provizyonlanan, Machine Config Operator (MCO) tarafından yönetilen işletim sistemi. 2) Cluster Operators: DNS, Network, Ingress, Authentication, Storage gibi tüm platform bileşenleri birer Kubernetes Operatörüdür. Cluster Version Operator (CVO), tüm kümenin sürüm yükseltmelerini tek bir bildirimle otomatik gerçekleştirir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift 4 architecture centers on: 1) Immutable RHCOS provisioned via Ignition and managed entirely by the Machine Config Operator (MCO). 2) Self-driving Platform Operators: Network, DNS, Ingress, and Storage are all managed by specialized operators. The Cluster Version Operator (CVO) orchestrates end-to-end zero-downtime cluster upgrades declaratively.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Kümedeki tüm platform operatörlerinin sağlık durumunu incele:
oc get clusteroperators

# Küme sürüm durumunu ve yükseltme geçmişini sorgula:
oc get clusterversion
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

CoreOS düğümlerine girip manuel olarak `apt-get` veya `yum install` yapamazsınız! Kök dosya sistemi salt-okunurdur. Tüm sistem ayarları `MachineConfig` CRD'leri üzerinden yapılmalıdır.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

Never attempt to manually patch RHCOS nodes via SSH. The root OS filesystem is read-only; all kernel tweaks must flow through `MachineConfig` manifests.

🎯 Özet Çıkarımlar

- RHCOS değiştirilemez işletim sistemi sayesinde düğümler asla 'drift' olmaz.
- Platform Operatörleri kümenin kendi kendini yönetmesini ve onarmasını sağlar.
- Tek tıkla tüm küme (işletim sistemi dahil) kesintisiz güncellenebilir.

🎯 Key Takeaways

- Immutable CoreOS prevents configuration drift across physical and virtual nodes.
- Platform operators manage cluster subsystems autonomously with self-healing loops.
- Enables seamless, one-click atomic updates spanning both K8s and the underlying OS.

39

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

`oc` Komut Satırı ve OpenShift Web Konsolu ile Görsel Yönetim

The `oc` CLI & Enterprise Web Console Experience

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

`oc` komut satırı aracı, `kubectl`'in siyah kuşak sahibi karate hocasıdır! `kubectl`'in tüm komutlarını anlar ama üstüne `oc new-app`, `oc login`, `oc project` gibi hayat kurtaran sihirli komutlar ekler. Web Konsolu ise bilim kurgu filmlerindeki kumanda merkezidir: Mikroservislerinizin durumunu birbiriyle konuşan renkli halkalar (Pod Donuts) halinde görsel olarak izlersiniz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The `oc` CLI is `kubectl` with a black belt in karate! It executes every single `kubectl` command natively, while introducing developer shortcuts like `oc new-app`, `oc status`, and `oc project`. Meanwhile, the OpenShift Web Console looks like a NASA mission control room: dynamic visual topology graphs display your Spring Boot pods as glowing, animated status rings!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`oc` CLI ile OpenShift'e özgü kaynaklar (`routes`, `buildconfigs`, `imagestreams`, `deploymentconfigs`) yönetilir. `oc project coffee-shop` komutu, `kubectl config set-context` komutunun yaptığı karmaşık isim alanı geçişini tek hamlede yapar. Web Konsolu iki görünüme sahiptir: 1) Developer Perspective (Kod odaklı, Git bağlantılı, görsel topoloji haritası), 2) Administrator Perspective (Düğümler, depolama, RBAC ve küme operatörleri).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The `oc` binary provides first-class support for OpenShift custom resource definitions (`routes`, `imagestreams`, `buildconfigs`). Running `oc project <name>` simplifies namespace switching. The Web Console features two purpose-built modes: 1) Developer Perspective (visual topology maps, Git import wizards, log streams), 2) Administrator Perspective (cluster health, CRDs, quotas, hardware nodes).

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. OpenShift kümesine giriş yap
oc login https://api.mycluster.openshift.com:6443 -u developer

# 2. Yeni bir proje (güçlendirilmiş namespace) oluştur ve geç
oc new-project coffee-shop

# 3. Projenin genel durumunu özetle
oc status

# 4. Pod ve servisleri listele
oc get all
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

OpenShift'te 'Project' aslında arka planda bir Kubernetes 'Namespace'idir. Ancak Project nesnesi ek olarak açıklama, görünen isim ve otomatik RBAC izinleri barındırır.

🇬🇧 WATCH OUT

In OpenShift, a 'Project' is a decorated Kubernetes Namespace with metadata annotations and pre-configured default RBAC roles.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- `oc` CLI `kubectl` ile %100 uyumludur ve ek kurumsal kolaylıklar sunar.
- Developer Web Konsolu mikroservis bağımlılıklarını görsel grafikle sunar.
- Tek bir komutla (`oc project`) ortamlar arasında zahmetsizce geçiş yapılır.

- The `oc` CLI is fully compatible with `kubectl` while adding high-productivity shortcuts.
- The visual topology console visualizes real-time microservice interactions intuitively.
- Switching environments is as simple as typing `oc project <name>`.

40

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

Routes vs Ingress: OpenShift Trafiği Nasıl Dışa Açar?

OpenShift Routes vs K8s Ingress: Edge TLS Demystified

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes dünyası henüz Ingress'i icat etmemişken, Red Hat mühendisleri 'Route' mekanizmasını geliştirmişti! K8s'te Ingress yazmak için önce gidip NGINX controller kurmanız, DNS kayıtlarıyla boğuşmanız gerekir. OpenShift'te ise tek bir komut verirsiniz (`oc expose svc coffee-service`), sistem size saniyesinde geçerli bir HTTPS adresi (`coffee-service-coffee-shop.apps.mycluster.com`) hediye eder!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Years before Kubernetes standardized Ingress, Red Hat engineered OpenShift Routes! In plain K8s, exposing an app requires installing an ingress controller, procuring domain names, and mapping DNS records. In OpenShift, you run ONE command (`oc expose svc coffee-service`), and OpenShift instantly creates a public, TLS-secured wildcard domain!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

OpenShift Route, küme girişindeki HAProxy tabanlı Router pod'ları tarafından yönetilir. Üç farklı TLS sonlandırma stratejisi sunar: 1) `Edge`: SSL sertifikası Route (HAProxy) seviyesinde sonlandırılır, arkadaki Spring Boot pod'una düz HTTP olarak iletilir (en yaygın ve hızlı model). 2) `Passthrough`: Şifrelenmiş SSL trafiği hiç açılmadan doğrudan pod'un içine kadar iletilir (uçtan uca şifreleme). 3) `Re-encrypt`: Route sertifikayı açar, inceler ve pod'a giderken farklı bir iç sertifikayla yeniden şifreler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

OpenShift Routes are fulfilled by HAProxy routers. Three TLS termination strategies are supported: 1) `Edge`: TLS terminates at the HAProxy router, proxying plain HTTP to backend pods (highest performance), 2) `Passthrough`: Encrypted TLS packets pass through untouched directly to the pod's embedded SSL engine, 3) `Re-encrypt`: The router decrypts external traffic, inspects it, and re-encrypts using an internal certificate before reaching the pod.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# coffee-route.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: coffee-route
  namespace: coffee-shop
spec:
  to:
    kind: Service
    name: coffee-service
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect # HTTP isteklerini otomatik HTTPS'e yönlendir!

# CLI ile tek komutta Route üretmek:
# oc expose service coffee-service --port=http
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

OpenShift aynı zamanda standart Kubernetes `Ingress` kaynaklarını da destekler (arka planda otomatik Route'a dönüştürür). Ancak kurumsal sertifika yönetimi için Route tanımlamak çok daha güçlü ve pratiktir.

🇬🇧 WATCH OUT

OpenShift automatically translates K8s Ingress objects into Routes under the hood, but native Routes provide finer-grained TLS and timeout controls.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Route, OpenShift'in yerleşik ve otomatik DNS destekli dışa açma mekanizmasıdır.
- `Edge`, `Passthrough` ve `Re-encrypt` ile esnek TLS mimarileri kurulur.
- HTTP trafiği `Redirect` politikasıyla otomatik olarak güvenli HTTPS'e yükseltilir.

- Routes provide out-of-the-box wildcard DNS and Layer 7 external routing.
- Flexible TLS termination models satisfy rigorous enterprise security compliance.
- Enforces automated HTTP-to-HTTPS redirects with a single line of YAML.

41

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

OpenShift Güvenliği: Security Context Constraints (SCC)

OpenShift Security: Security Context Constraints (SCC)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Neden Docker'da veya Minikube'da şakır şakır çalışan Spring Boot imajınız OpenShift'e atınca `Permission Denied` hatasıyla yere yapışır? Çünkü OpenShift aşırı titiz bir banka müdürü gibidir: 'Sen bana sormadan kendine root (UID 0) yetkisi vermişsin, hayatta izin vermem!' der. OpenShift her projeye rastgele bir kullanıcı kimliği (örneğin UID 1000620000) atar ve konteyneri o kimlikle çalışmaya zorlar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Why does a Docker image that runs perfectly on Minikube crash with `Permission Denied` on OpenShift? Because OpenShift enforces strict Security Context Constraints (SCC)! It says: 'You dare run as root (UID 0)? Forbidden!' OpenShift dynamically assigns a random high-numbered user ID (e.g. UID 1000620000) to every project and forces your container into it!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Security Context Constraints (SCC), Pod'ların kümede hangi ayrıcalıklarla çalışabileceğini belirleyen OpenShift güvenlik motorudur (K8s PodSecurity'nin atasıdır). Varsayılan SCC `restricted-v2`'dir: 1) Konteynerin `root` olarak çalışmasını engeller, 2) Rastgele tahsis edilen bir UID aralığı zorunlu kılar, 3) Host dizinlerini (`hostPath`) bağlamayı yasaklar. Çözüm: Dockerfile'da dosyaların grup sahipliğini `root (GID 0)` grubuna verip grup yazma izni tanımaktır (`chown -R 0 /app && chmod -R g=u /app`).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Security Context Constraints (SCC) control pod runtime permissions in OpenShift. The default `restricted-v2` SCC prohibits root execution, blocks hostPath mounts, drops capabilities, and dynamically allocates an arbitrary UID from the project's allocated UID range. Enterprise Dockerfiles comply by setting folder permissions to GID 0: `chown -R 0 /app && chmod -R g=u /app`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# OpenShift Uyumlu Dockerfile Katmanı:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/app.jar app.jar

# OpenShift rastgele bir UID ile çalıştırsa bile grubun (GID 0) yazma hakkı olsun:
RUN chown -R 0 /app && \
    chmod -R g=u /app

# Asla ROOT olarak çalıştırma!
USER 10001
ENTRYPOINT ["java", "-jar", "app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Acemi mühendisler hatayı çözmek için pod'a `oc adm policy add-scc-to-user anyuid -z default` vererek güvenlik duvarını delerler. Bu kurumsal denetimlerde (audit) affedilmeyen bir güvenlik ihlalidir! İmajınızı OpenShift standartlarına uygun hale getirin.

🇬🇧 WATCH OUT

Never grant `anyuid` SCC to bypass permission errors in production! Fix your container image file permissions to support arbitrary UIDs instead.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- SCC, konteynerlerin sunucu işletim sistemini ele geçirmesini kesin olarak engeller.
- OpenShift rastgele tahsis edilmiş dinamik UID'ler kullanarak güvenliği artırır.
- Uygulama klasörlerine GID 0 (`root group`) okuma/yazma izni verilmelidir.
- SCC prevents malicious containers from executing privileged host escalations.
- Dynamic random UIDs ensure isolation even if containers share hardware nodes.
- Supporting OpenShift requires granting GID 0 read/write access in your Dockerfile.

42

BÖLÜM: RED HAT OPENSHIFT (OCP) – KURUMSAL KUBERNETES | MODULE: RED HAT OPENSHIFT (OCP) – ENTERPRISE KUBERNETES

Source-to-Image (S2I) ile Spring Boot Kodunu Otomatik Dağıtma

Deploying Spring Boot via Source-to-Image (S2I) & Binary Builds

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dockerfile yazmayı bile bilmeyen bir yazılımcı düşünün! OpenShift'e der ki: 'Al abi benim GitHub depomun linki burada, Java kodlarım içinde.' OpenShift sihirbaz gibi devreye girer: Kodu çeker, içine Maven ve JDK koyup derler, JAR'ı üretir, hafif bir runtime konteynerine koyar ve tek bir komutla canlıya alır! Buna S2I (Source-to-Image) denir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine a developer who has never written a Dockerfile in their life! They say to OpenShift: 'Here is my GitHub repository URL containing Java code.' OpenShift acts like a wizard: it clones the repo, provisions a Maven builder, compiles the JAR, bakes a lean container image, and deploys it in one breath! That is S2I (Source-to-Image)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

S2I mimarisi üç nesne ile çalışır: 1) `BuildConfig` (Derleme stratejisini, Git kaynağını ve tetikleyicileri tanımlar), 2) `ImageStream` (İmaj sürümlerini sanal olarak etiketleyen ve yeni imaj geldiğinde otomatik dağıtım tetikleyen soyutlama), 3) Builder Image (Red Hat'in sağladığı optimize `ubi8/openjdk-21`). Ayrıca lokaldeki `.jar` dosyasını doğrudan göndermek için 'Binary Build' (`oc start-build --from-file=target/app.jar`) yöntemi de sıklıkla kullanılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

S2I automates image creation via: 1) `BuildConfig` (defines Git source, webhooks, and builder images), 2) `ImageStream` (tracks container image metadata and triggers automated rollout upon updates), 3) Curated Red Hat OpenJDK builder images. Alternatively, teams use Binary Builds to push locally compiled JAR binaries directly into OpenShift build pods.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# 1. Java Git Deposundan Doğrudan Uygulama Üret (S2I):
oc new-app registry.access.redhat.com/ubi8/openjdk-21~https://github.com/my-org/spring-boot-k8s-eli5.git \
  --name=coffee-s2i-app

# 2. Derleme Loglarını Canlı Takip Et:
oc logs -f bc/coffee-s2i-app

# 3. Servisi Dış Dünyaya Aç:
oc expose svc/coffee-s2i-app
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Büyük kurumsal projelerde S2I küme üzerindeki CPU ve bellek kaynaklarını yoğun tüketebilir. Çok büyük ekiplerde derleme işlemini GitHub Actions gibi harici bir CI aracında yapıp OpenShift'e sadece hazır imajı iletmek daha verimlidir.

🇬🇧 WATCH OUT

Compiling multi-gigabyte Maven builds inside the cluster via S2I consumes cluster compute. For large teams, offloading compilation to GitHub Actions is best practice.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- S2I geliştiricilerin Dockerfile yazma zorunluluğunu ortadan kaldırır.
- ImageStreams yeni bir imaj etiketi geldiğinde otomatik dağıtımı tetikler.
- Kurumsal Red Hat UBI (Universal Base Image) güvenli temel sağlar.

- S2I abstracts container packaging away from application developers entirely.
- ImageStreams automatically trigger rolling deployments when new tags appear.
- Red Hat Universal Base Images (UBI) deliver enterprise-grade, certified runtimes.